Ders 11 Genişletilmiş Notu: Bağlı Liste Kodlama (Ekleme, Yazdırma, Silme), Yapı Hizalaması, Bit Alanları, Union ve Özyineleme

Ders 11 Genişletilmiş Notu: Bağlı Liste Kodlama (Ekleme, Yazdırma, Silme), Yapı Hizalaması, Bit Alanları, Union ve Özyineleme

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Bağlı Listeye Kısa Tekrar
- 2. Düğüm Yapısı ve Listede Yapılacak İşlemler
- 3. Listeyi Yazdırma
- 4. Düğüm Oluşturma: Neden Yerel Değişken Olmaz, Neden malloc?
- 5. Bütün Listeyi Silme: İlk Anlatım
- 6. Sona Ekleme ve İlk Denemedeki Hata
- 7. head Ne Zaman Değişir?
- 8. Programı Çalıştırmak: Yazdırma Döngüsünün Düzeltilmesi
- 9. deleteList ve main'deki head Sorunu
- 10. Slayttaki Çalışan (Employee) Örneği
- 11. Yapı Hizalaması (Structure Alignment)
- 12. Bit Alanları (Bit Field)
- 13. Birlikler (Union)
- 14. Özyinelemeye Giriş: Faktöriyel
- 15. Doğrudan ve Dolaylı Özyineleme
- 16. Stack Overflow Nasıl Oluşur?
- 17. Stack'i Deneyle Görmek
- 18. Fibonacci: Döngüyle ve Özyinelemeyle
- 19. Üs Alma: Özyinelemenin İşe Yaradığı Bir Örnek

1. Bağlı Listeye Kısa Tekrar

Hoca, önceki derste bağlı listelere (linked list) giriş yapıldığını hatırlatarak başladı. Bugün bu konuyla ilgili kod yazılacaktı. Kod yazarken bir zorlukla karşılaşılacağını, bu zorluğun nasıl aşılacağının ise özyineleme (recursion) konusu görülünce konuşulacağını da baştan söyledi.

Bağlı listede her düğüm iki parçadan oluşur. Birincisi veridir (data). Verinin türü herhangi bir şey olabilir, "ne olduğu çok mühim değil". İkincisi bir adres tutan işaretçidir. Tek yönlü bağlı listede bu işaretçi bir sonraki düğümün adresini tutar ve adına `next` denir. Listedeki her parçaya düğüm (node) denir.

İlk düğümün özel bir adı vardır: `head`. Program aslında bir düğüm oluşturup bu düğümün işaretçisini `head` adlı bir değişkende saklar. Bir bağlı liste oluşturmak, aslında `head` değişkenini tutmak demektir. Çünkü `head` biliniyorsa bütün düğümlere adım adım, sırayla erişilebilir.

Son düğümün ardından başka düğüm gelmez. Tek yönlü ve dairesel olmayan bir listede son düğümün adres alanı `NULL` değerini alır.

Bazen son elemana hızlıca gitmek istenir. Bunun tipik sebebi sona eleman eklemektir. Her ekleme için baştan başlayıp düğümleri tek tek gezerek sonu bulmak mantıklı değildir. Bu yüzden son düğüm `tail` adlı ayrı bir işaretçide tutulabilir.

Hoca ardından listenin başka biçimlerini saydı:

- **Dairesel tek yönlü liste (circular single linked list):** Son düğümün adres alanına `NULL` yerine ilk düğümün adresi yazılır.
- **Çift yönlü liste (double linked list):** Her düğümde veri ve iki işaretçi vardır: bir sonraki düğüm için `next`, bir önceki düğüm için `prev`.
- **Dairesel çift yönlü liste:** Son elemana erişmek daha kolaydır, ayrı bir işaretçi tutmak gerekmez. Çünkü `head`'in bir öncesi son düğümdür.

Hoca bu tür algoritmaların literatürde genellikle iç içe, özyinelemeli fonksiyonlarla yazıldığını, ama bugün klasik `for` ve `while` döngüleriyle kurulacağını söyledi. Soruna farklı bakış açısı daha sonra gelecekti. Yapının kendisinde karmaşık bir şey yoktu. Fikir yürütmek gereken yer ekleme ve silmeydi.

2. Düğüm Yapısı ve Listede Yapılacak İşlemler

Hoca veriyi şimdilik tek bir tam sayı olarak aldı ve ayrı bir yapı kullanmadı:

```
typedef struct node {
    int data;
    struct node *next;
} node;
```

Listede kaç eleman olacağı baştan belli değildir ve böyle bir kısıt da yoktur. Bilgisayarın belleği ne kadar izin veriyorsa o kadar düğüm eklenebilir. Düğümler eklenip birbirine bağlandıktan sonra liste baştan sona gezilebilir, elemanları ekrana yazdırılabilir, listeden eleman çıkarılabilir, başa, sona ya da araya eleman eklenebilir. Bunların her biri farklı bir iş olduğundan fonksiyon olarak yazılır.

3. Listeyi Yazdırma

Bütün listeyi yazdırmak için fonksiyona yalnızca başlangıç düğümünü vermek yeterlidir. Başlangıç düğümü biliniyorsa `next`, `next`, `next` diyerek diğerlerine geçilir. Eleman sayısı bilinmediği için sayaçlı bir döngü değil, `while` ya da `do-while` gibi bir döngü gerekir.

```
void printList(node *head) {
    while (head != NULL) {
        printf("%d ", head->data);
        head = head->next;
    }
}
```

`head` bir işaretçi olduğu için alanına ok işaretiyle (`->`) erişilir. Veri yazıldıktan sonra bir sonraki düğüme geçmek için `head` güncellenir. Döngü `NULL` değere ulaşılınca durur.

Liste oluşturmak ise listeye düğüm eklemek demektir. Hoca buradan düğüm eklemeye geçti.

4. Düğüm Oluşturma: Neden Yerel Değişken Olmaz, Neden malloc?

Düğüm eklemek için önce bir düğüm oluşturmak, sonra belli bir mantığa göre (başa, ortaya ya da sona) onu listeyle ilişkilendirmek gerekir. Hoca önce bilerek şöyle bir kod yazdı ve "Burada atladığım bir şey var mı?" diye sordu:

```
void addNode(node *head) {   /* YANLIS: derste hata gostermek icin yazildi */
    node n;
    while (head != NULL)
        head = head->next;
    head = &n;
}
```

Fikir şuydu: Liste bitene kadar git, gidebildiğin yere yeni düğümün adresini yaz. Cevap sınıftan geldi ve hoca bunu doğruladı. Bir yapı değişkeninin bir `int` değişkenden farkı yoktur. Fonksiyon açıldığında `n` stack bölgesinde üretilir. Fonksiyon bitince o değişkenin kalıcılık hakkı kalmaz. Düğüm "fonksiyon yaşayana kadar garanti"dir. Fonksiyon bittikten sonra o yerde başka bir fonksiyonun başka bir değişkeni tanımlanır. Dolayısıyla bu yolla düğüm üretilemez.

Kalıcı bir liste oluşturulacaksa düğümler heap bölgesinde olmalıdır. Heap bölgesinde değişken olması da dinamik bellek yönetimi, yani `malloc` gibi bir fonksiyon kullanmak demektir. Hoca işleri ayırdı. Ekleme fonksiyonunun hedefi düğümü listeye eklemektir, düğüm üretmek ayrı bir fonksiyonun işidir:

```
node *createNode(void) {
    int d;
    node *n;
    printf("Veri girin: ");
    scanf("%d", &d);
    n = (node *) malloc(sizeof(node));
    n->data = d;
    n->next = NULL;
    return n;
}
```

Hoca yerel değişkenin adı `data` alanıyla karışmasın diye `d` dedi. Dönüş değeri `malloc` ile oluşturulduğu için fonksiyon bittikten sonra da yaşamaya devam eder.

`next` satırı ilk yazımda yoktu. Hoca kısa süre sonra geri dönüp bunu ekledi. Düğüm oluşturulduğunda şimdilik tek başına, tekil bir düğümdür. İleride baştaki `head` de olabilir, sondaki `tail` de, aradaki herhangi bir düğüm de. O yüzden `NULL` yapmak için en doğru yer bu fonksiyondur. `NULL` yapılmazsa bellekte kalan çöp değerden dolayı `next` alanında sayısal bir değer bulunabilir ve bu değer yüzünden liste karışır.

> **Hocanın vurgusu:** Bir bağlı liste oluşturuyorsak bu listenin düğümleri "kesinlikle ama kesinlikle" dinamik bellek yönetimi metotlarıyla oluşturulacaktır.

5. Bütün Listeyi Silme: İlk Anlatım

Bütün listeyi silmek demek, listeyi `free` etmek demektir. Kaç tane düğüm oluşturulduysa o kadar `free` gerekir. Hoca burada iki yanlış yolu eledi.

Birincisi ilk düğümü doğrudan silmektir. İlk eleman silinirse ikinci, üçüncü elemana giden bağlantı kaybolur. İkincisi her defasında sona gidip sondan silmektir. Bu da çok zahmetlidir. n düğüm için önce n adım, sonra $n-1$ adım gidilir ve bu böyle bire kadar sürer. Hoca bunu "n^2 defa işlem" diye özetledi.

> **Ders sonrası not:** Toplam kesin olarak $n + (n-1) + \ldots + 1 = n(n+1)/2$ eder. Yani işlem sayısı n^2 değil, $n^2/2$ civarındadır. Hocanın söylediği, büyüme hızının n^2 mertebesinde olmasıdır.

Çözüm baştan silmek ve silmeden önce gerekeni bir yerde yedeklemektir. Bunun için `tmp` adlı geçici bir işaretçi kullanılır. Hoca önce şu sırayı önerdi: `head`'in sonrakini `tmp`'ye al, `head`'i `free` et, `tmp`'yi `head` yap.

```
while (head != NULL) {
    tmp = head->next;
    free(head);
    head = tmp;
}
```

Sınıftan gelen öneriyle ikinci bir sıra da yazıldı: Önce `tmp`'ye `head`'in kendisini al, `head`'i bir sonrakine ilerlet, sonra `tmp`'yi `free` et.

```
while (head != NULL) {
    tmp = head;
    head = head->next;
```

```
    free(tmp);
}
```

İkisi de doğrudur. Ortak nokta şudur: Bir düğüm `free` edildikten sonra o düğüm artık yoktur, `next` alanı okunamaz. Bu yüzden bağlantı `free`'den önce saklanmış olmalıdır. Hoca ikinci sırada yaşananı adım adım şöyle anlattı: `tmp` ile önce ilk düğüm yedeklenir, `head` bir sonrakine alınır, `tmp`'nin gösterdiği düğüm serbest bırakılır, döngü başa döner.

Listeden aradaki bir elemanı silmek ise ayrı bir konudur. Bunun bir kısmı bu dersin, bir kısmı veri yapıları dersinin konusudur.

6. Sona Ekleme ve İlk Denemedeki Hata

`main` içinde ilk düğüm `head` olarak oluşturulur. Sonraki bütün ekleme ve yazdırmalar bu `head` üzerinden yapılır:

```
node *head = createNode();
addLast(head);
printList(head);
```

Hoca, ekleme fonksiyonunu ilk hâliyle bir düğümlü liste üzerinde kağıtta izleyerek hatayı gösterdi. İlk hâlde döngü koşulu `head != NULL`, son atama da `head = n` idi.

1. Listede bir düğüm var. `head` `NULL` değil, döngüye girilir.
2. `head = head->next` yapılır. İlk düğümün `next`'i `NULL` olduğundan `head` artık boş bir yeri gösterir.
3. `createNode` ile yeni düğüm elde edilir ve `head = n` yapılır.
4. Yeni düğüm, mevcut düğümün `next`'iyle hiç ilişkilendirilmemiştir. Arada bağlantı kurulmaz.

"İşin daha da kötüsü" de şudur: `head` buraya parametre olarak gelmiştir. `main`'deki `head` bir yeri gösterir, fonksiyondaki yerel `head` başka bir yeri. Fonksiyon çalışır ve işe yaramaz.

Doğrusu döngüyü son düğümde durdurmaktır. Koşul `head != NULL` değil, `head->next != NULL` olmalıdır:

```
void addLast(node *head) {
    node *n;
    while (head->next != NULL)
        head = head->next;
    n = createNode();
    head->next = n;
}
```

Yine bir düğümlü listede izlenirse: `head->next` `NULL`'dan farklı olmadığı için döngüye girilmez. Yeni düğüm oluşturulur ve `head->next = n` ile bağlanır. Bu fonksiyon yerel `head`'i değiştirir ama `main`'deki `head` aynı noktayı göstermeye devam eder. Sonuçta dizinin sonuna bir eleman eklenmiş olur.

Hoca bu eklemenin eksik bir yanını da belirtti: `head`'in `NULL` olma durumu ayrıca düşünülmelidir. Bu örnekte `main` en az bir düğüm oluşturduğu için sorun çıkmaz. Ayrıntısını sonraki derse bıraktı.

Hoca, elle yazmak yerine bir `do-while` döngüsüyle ekleme yaptırdı. Her eklemeden sonra kullanıcıya devam etmek isteyip istemediği sorulur. Cevap evet olduğu sürece döngü sürer.

Soru: `createNode`'u ayrıca çağırmak gerekir mi? **Cevap:** Hayır, `addLast` onu zaten kendi içinden çağırıyor.

7. head Ne Zaman Değişir?

Hoca bu noktada önemli bir ayrım yaptı. `addLast`, yapısı gereği listenin sonuna eleman eklediği için `head`'i değiştirmez. `addFirst` öyle değildir. Başa eleman eklenince `head` değişir ve bu değişikliğin `main`'e bildirilmesi gerekir. Yoksa `main` `head`'i hâlâ ilk oluşturulan düğüm olarak bilir. Bu da bir sorundur.

Araya eklemeyi hoca `addLast` gibi düşünülebilecek bir işlem olarak andı, çünkü `head` değişmez. Ama araya eklemek sona eklemek kadar kolay değildir. Mevcut düğüm ile bir sonraki düğüm arasında yeni düğüm için ilişkilendirme yapmak gerekir.

Bir soru üzerine hoca, `addLast`'ın işleyişini bir kez daha özetledi. İkinci elemanı eklerken birincinin sonuna gidilir. Üçüncüyü eklerken baştan ikincinin sonuna gidilir. Böylece liste artarak genişler. `head`'in kaybedileceği bir durum yoktur. `head` ancak listenin başına eleman eklenince ya da başından eleman silinince değişir.

8. Programı Çalıştırmak: Yazdırma Döngüsünün Düzeltilmesi

Hoca derlemede bir uyarı gördü: `createNode` çağrıldığı yerden önce bildirilmemişti. Ardından programı çalıştırıp 4, 3 ve 7 girdi. Ekranda yalnızca 4 ve 3 göründü, 7 yazılmadı.

Hoca, listede en az bir eleman olduğu için yazdırma döngüsünün koşulunu o sırada `head->next != NULL` diye değiştirmişti. Bu yüzden son düğüme gelindiğinde döngü onu yazmadan bitiyordu. İlk çözüm, döngüden sonra son düğümün verisi için fazladan bir `printf` eklemek oldu. 1, 2, 3 girişiyle çıktı doğru görüldü.

Soru: Neden `head->next`'i `NULL` ile karşılaştırıyoruz? Doğrudan `head`'i kontrol etsek, sondaki ek yazdırmaya gerek kalmaz, değil mi? **Cevap:** Evet, bu da olur. Önce veri yazılır, sonra `next`'e geçilir, `head` `NULL`'dan farklı olduğu sürece döngü sürer. Son düğüm de döngü içinde yazılır.

Soru: Öyle yaparsak hata olur demiştiniz, hata olur mu? **Cevap:** Burada hata yok. Yalnız sıralamaya dikkat etmek gerekir. Önce `NULL` kontrolü yapılıp sonra veriye erişilir. Sıra ters çevrilirse, `NULL` kontrolü yapılmadan alana erişmeye çalışılacağı için hata alınır.

Hoca buna ek olarak bir de boş liste kontrolü yazdı. `head` `NULL` ise ekrana liste boş diye yazılıp fonksiyondan çıkılır. `return` edildiği için `else` yazmaya gerek yoktur:

```
void printList(node *head) {
    if (head == NULL) {
        printf("empty list\n");
        return;
    }
    while (head != NULL) {
        printf("%d ", head->data);
        head = head->next;
    }
    printf("\n");
}
```

9. deleteList ve main'deki head Sorunu

Silme fonksiyonu önce `void` olarak yazıldı. Ayrıca `head != NULL` ya da `head->next != NULL` gibi farklı koşullarla farklı kodlar yazılabileceğini, tek bir doğru yazımın olmadığını hoca özellikle söyledi. `main`'de sırayla listeyi yazdır, sil, tekrar yazdır dendi. 1, 2, 3, 4, 5 girildi ve ilk yazdırma çalıştı, ama sonrasında program çöktü: "Bir yerde bir açığımız var demek ki. Ne oldu? Patlattık."

Sebep şudur: Düğümler fonksiyon içinde tek tek silindi ve fonksiyondaki `head` güncellendi. Ama bu güncelleme yalnızca fonksiyonun içinde kaldı. `main`'deki `head` hâlâ başlangıç düğümünün adresini gösteriyor, oysa o düğüm artık yok. `main`'deki `head`'in `NULL` olarak güncellenmesi gerekir.

> **Ders sonrası not:** Bu durum `void` sürümle gcc ve AddressSanitizer altında denendi: İkinci yazdırma `heap-use-after-free` hatasıyla durdu. Serbest bırakılmış belleği okumanın sonucu büyük olasılıkla makineye göre değişir. Hoca derste çöküşü de listenin kendi içinde dönüyor gibi görünmesini de bu yüzden gördü.

Hoca iki yol gösterdi. Birincisi `head`'in kendisini değiştirmektir. `head` zaten bir işaretçi olduğundan, onu fonksiyondan değiştirebilmek için ikinci dereceden işaretçi gerekir. Hoca bunu yazmaya başladı, ama "bununla kafanı karıştırmayayım" diyerek geri aldı. İkinci ve daha kolay yol, fonksiyondan `NULL` döndürüp `main`'de `head`'e atamaktır:

```
node *deleteList(node *head) {
    node *tmp;
    while (head != NULL) {
        tmp = head->next;
        free(head);
        head = tmp;
    }
    return NULL;
}
/* main icinde: head = deleteList(head); */
```

Bu hâliyle birinci yazdırma listeyi, ikinci yazdırma “empty list” çıktısını verdi. Hoca bunun programlama olarak doğru olduğunu ama normalde silmenin böyle yazılmadığını söyledi. Olması gereken, fonksiyonun `void` kalmasıdır. Bunun örneğini özyinelemeyle birlikte yapacaklarını belirtti.

> **Hocanın vurgusu:** Birinci düğümden silmede ya da birinci düğüme eklemede `head` değiştiği için ikinci dereceden işaretçiye mutlaka ihtiyaç duyuyoruz.
>
> **Ders sonrası not:** Hocanın cümlesi, `void` yazım için geçerlidir. Bir önceki kodun gösterdiği gibi, yeni `head`’i döndürüp atamak da aynı sorunu çözer. Yani `node **` tek yol değildir.
>
> **Ek bilgi (derste anlatılmadı):** Derste yarım bırakılan ikinci dereceden işaretçili yazım aşağıdadır. Kod kısa nottan alındı, derste yazılmadı. Tam programla `gcc -Wall` altında uyarısız derlendi ve AddressSanitizer ile çalıştırıldı; bellek hatası çıkmadı, çağrıdan sonra `head` `NULL` oldu.

```
void deleteList2(node **h) {
    node *t;
    while (*h != NULL) {
        t = *h;
        *h = (*h)->next;
        free(t);
    }
}
/* cagri: deleteList2(&head); */
```

Buraya kadar yazılan parçaların birleşmiş hâli aşağıdadır.

> **Ders sonrası not:** Aşağıdaki program altyazıdan birleştirildi, ekrandaki kodun birebir kopyası değildir. Bildirim uyarısı çıkmasın diye fonksiyonlar çağrılmadan önce tanımlandı. Devam sorusunun metni büyük olasılıkla derstekinden farklıdır. Program `gcc -Wall` ile uyarısız derlendi. 1, 2, 3, 4, 5 girişiyle önce `1 2 3 4 5`, silmeden sonra `empty list` yazdı; AddressSanitizer bellek hatası bildirmedi.

```
#include <stdio.h>
#include <stdlib.h>

typedef struct node {
    int data;
    struct node *next;
} node;

node *createNode(void) {
    int d;
    node *n;
    printf("Veri girin: ");
    scanf("%d", &d);
    n = (node *) malloc(sizeof(node));
    n->data = d;
    n->next = NULL;
    return n;
}
```

```c
void addLast(node *head) {
    node *n;
    while (head->next != NULL)
        head = head->next;
    n = createNode();
    head->next = n;
}

void printList(node *head) {
    if (head == NULL) {
        printf("empty list\n");
        return;
    }
    while (head != NULL) {
        printf("%d ", head->data);
        head = head->next;
    }
    printf("\n");
}

node *deleteList(node *head) {
    node *tmp;
    while (head != NULL) {
        tmp = head->next;
        free(head);
        head = tmp;
    }
    return NULL;
}

int main(void) {
    int devam;
    node *head = createNode();
    do {
        addLast(head);
        printf("Devam (1/2)? ");
        scanf("%d", &devam);
    } while (devam == 1);
    printList(head);
    head = deleteList(head);
    printList(head);
    return 0;
}
```

10. Slayttaki Çalışan (Employee) Örneği

Hoca bu bölümde slayttaki hazır bir kodu okudu. Kodun şu ana kadar görülenlerin çoğunu kapsadığını söyledi.

10.1 Yapı ve scanf'te & Kullanımı

Veri bu kez tek bir tam sayı değil, ad, soyad, kimlik numarası ve maaştan oluşan bir `employee` yapısıdır. Düğüm yapısı bu yapıyı veri olarak içerir ve yanında `next` tutar. Yani iç içe (nested) bir yapı vardır.

Düğüm oluştururken önce `malloc` ile yer açılır, sonra ad, soyad, kimlik ve maaş kullanıcıdan alınır, `next` `NULL` yapılır. Hoca burada `scanf` için sınıfa & sorusunu sordu. Ad ve soyad karakter dizisidir, & kullanılmaz. Kimlik ve maaş ise `int` ve `float` gibi skaler değerlerdir, düğümün içindeki bu alanların adresi bilinmediği için & kullanılır.

> **Hocanın vurgusu:** `scanf`'te karakter dizileri için `&` kullanılmaz, skaler alanlar için kullanılır. "Buna dikkat edelim."
>
> **Ders sonrası not:** Hoca dizi için "kendisi zaten bir pointer tuttuğu için" dedi. Dizi ayrı bir işaretçi tutmaz. gcc 15.2 ile 64 bit Linux'ta denendi: `char name[20]` için `sizeof` 20, `char *` için 8 çıktı. Büyük olasılıkla doğru ifade, dizi adının ifade içinde ilk elemanın adresine dönüşmesidir. `scanf`'teki sonuç aynıdır.

10.2 Global head

Örnekte hocanın "çok da kullanmanızı tercih etmediğim" dediği bir şey vardır: `head`, `main`'in üzerinde global değişken olarak tanımlanmıştır. Amaç kolaylıktır. Az önce `deleteList` ve `addFirst`'te gördüğümüz sorun burada ortaya çıkmaz, çünkü her fonksiyon aynı değişkene erişir. Global değişken program kapanana kadar yaşar, stack derdi yoktur.

> **Hocanın vurgusu:** Global değişkenin kendine göre yan tesirleri vardır. Olması gereken her zaman parametre geçişidir. Bu örnekteki kolaylık yanıltmamalıdır.

10.3 Yazdırma

Örnekte iki yazdırma fonksiyonu vardır. Birincisi tek bir `employee` yapısının kimlik, ad ve maaşını yazar. Bu fonksiyon bağlı listeden bağımsızdır, bir diziyle de kullanılabilir.

Asıl liste yazdırma fonksiyonu `for` ile yazılmıştır: `head`'den başla, `j`'yi sıfırla, `p` `NULL`'dan farklı olduğu sürece her kişinin sırasını ve bilgilerini yaz, `p`'yi `next`'e ilerlet. Hoca bunun `for`'un böyle de kullanılabileceğini göstermek için yazıldığını, kendisinin olsa `while` kullanacağını söyledi. Fonksiyon parametre almaz, çünkü `head` globaldir. Aynı şekilde `head` değişse bile bir değer döndürmesine gerek olmaz.

10.4 Liste Elemanı Oluşturma

Önce `malloc` yapılır ve hata yoksa bilgiler alınır. Ad okunurken `&` yoktur. Kimlikte ise kod şöyle bir yol izlemiştir: Değeri önce `i` gibi geçici bir değişkene okuyup sonra yapının alanına aktarmıştır. Hoca bunun öğrencilerde de çok görüldüğünü söyledi: "`&` kullanacak mıydım, hatırlayamadım, geçici değişkene alayım." Bu çalışır, ama alanın adresini doğrudan vermek de mümkündür.

> **Ders sonrası not:** Hoca bu aktarmaya "deep copy" dedi. `int` atamasında işaretçi takibi yoktur, yapılan sıradan bir değer kopyalamadır. Derin kopya terimi, işaretçinin gösterdiği verinin de kopyalandığı durumlar için büyük olasılıkla daha uygundur.

10.5 Sona Ekleme

Bu kodun yaklaşımı hocanın yazdığından biraz farklı ama doğrudur. Önce bir düğüm oluşturulur. `head` `NULL` ise henüz listede hiç eleman yoktur, `head = e` yapılır ve fonksiyondan çıkılır. Aksi hâlde listenin sonuna gidilir.

Burada `head` önce `p`'ye yedeklenir. Çünkü `head` global olduğundan `head = head->next` yazılırsa global `head` bundan etkilenir. `p->next` `NULL`'dan farklı olduğu sürece `p = p->next` yapılır, sonra yeni düğüm sona bağlanır. Hoca mantığın basitliğinin global değişken sayesinde olduğunu, bu yüzden bu örneği çok tavsiye etmediğini tekrar etti.

10.6 Bir Düğümü Silme

Hoca silme mantığını tahtada çizerek anlattı. Bir kişinin adı ya da kimliği verilip silinmesi istendiğinde önce o düğümün işaretçisi bulunur, sonra bu işaretçi silme fonksiyonuna verilir. İşaretçi gerekir, çünkü mesele bir isim bilgisini silmek değil, o düğümü tamamen listeden çıkarmaktır. Kodda silinecek işaretçinin adı `gunner`'dır ve bulunmuş olduğu kabul edilir.

İki durum vardır:

- **Silinecek düğüm `head` ise:** `head = gunner->next` yapılır ve düğüm `free` edilir. Bu kadar basit olması `head`'in global olması sayesindedir.

- **Arada ya da sonda ise:** Silinecek düğümün bir öncekini bulmak gerekir. Çünkü önceki düğüm ile sonraki düğüm birbirine bağlanacaktır.

Öncekini bulan döngü gövdesi boş bir `for`'dur. `p` `NULL`'dan farklı ve `p->next` `gunner`'dan farklı olduğu sürece `p`'yi ilerletir. Çıkışta `p` `NULL` ise eleman bulunamamıştır. Değilse `p->next = p->next->next` yapılır, yani önceki düğüm sonrakine bağlanır, sonra düğüm `free` edilir.

Hoca, `head` global olmasaydı ne yapılacağını da söyledi: Fonksiyon `head`'i parametre olarak alır. `gunner == head` ise `head = gunner->next` yapılır ve yeni `head` döndürülür. Fonksiyonu çağıran yer de bu değeri yeniden atar.

```
node *deleteNode(node *head, node *gunner) {
    node *p;
    if (gunner == head) {
        head = gunner->next;
        free(gunner);
        return head;
    }
    for (p = head; p != NULL && p->next != gunner; p = p->next)
        ;
    if (p == NULL) {
        printf("Eleman bulunamadi\n");
        return head;
    }
    p->next = p->next->next;
    free(gunner);
    return head;
}
```

10.7 İsme Göre Arama

Arama `head`'den başlar ve `NULL` olana kadar gezer. Bulunulan düğümün adı aranan adla karşılaştırılır. Sonuç sıfırsa iki ad aynıdır ve düğüm döndürülür. Liste biterse `NULL` döner, yani bu adla bir eleman yoktur.

> **Hocanın vurgusu:** Karşılaştırma `strcmp` ile yapılmalıdır. `==` operatörü bunun için çalışmaz.

10.8 Bir Düğümden Sonra Ekleme

Son fonksiyon iki işaretçi alır. Önce ikisinin de `NULL` olmadığından emin olunur. İkisi birbirine eşitse ya da biri zaten diğerinin hemen arkasındaysa ekleme yapılmaz. Listede başa dönüp arama yapmaya gerek yoktur, çünkü düğümün yeri bellidir.

Hocanın okuduğu atamalar şunlardır: "P'nin next'i eşittir Q'nun next'i olsun, P de Q'nun next'i olsun."

```
p->next = q->next;
q->next = p;
```

Bu atamalar `P` düğümünü `Q`'nun arkasına koyar. Fonksiyonun görevi de "P'yi Q'dan sonra ekle" diye okunmuştur. Kontrol koşullarındaki "Q'nun next'i P ise ekleme yok" ifadesi de bununla uyumludur, çünkü `P` zaten `Q`'nun arkasındadır.

> **Ders sonrası not:** Hoca tahtada adları ters kullandı: `P`'nin listede yeri belli, `Q` listeye girecek eleman dedi ve "P'den sonra Q elementini eklemiş oluyoruz" diye bitirdi. Altyazıdaki atamalar ve koşullar büyük olasılıkla slayttaki gibidir ve yeni düğüm `P`, listedeki düğüm `Q`'dur. Adlar ne olursa olsun kalıp aynıdır: Önce yeni düğümün `next`'i ayarlanır, sonra önceki düğüm yeni düğüme bağlanır. Sıra ters çevrilirse sonraki düğümün adresi kaybolur ve yeni düğüm kendisini gösterir. Bu gcc ile denendi.

10.9 Slayttaki main ve Çalışma Önerisi

Slayttaki `main` önce iki eleman ekler ve listeyi yazdırır, sonra bir eleman daha oluşturup bir düğümün ardına ekler ve listeyi tekrar yazdırır. Hoca bu kodun ders notlarında olduğunu söyledi ve konu çalışılırken iki şey yapılmasını önerdi. Birincisi, kodu `head` global olmadan, `main` içinde tanımlayıp parametre aktararak yeniden yazmak. İkincisi, ekleme ve silmeleri kodu izlerken tahtadaki gibi çizerek takip etmek: onun `next`'ini ona bağla, bunu buna bağla.

> **Hocanın vurgusu:** Bu işlemlerde sıralama, yani işlem adımları önemlidir.
>
> **Ders sonrası not:** Aşağıdaki program anlatımdan kuruldu, slaytın birebir kopyası değildir. Ad alanları, dizi boyutları ve yazdırma biçimi büyük olasılıkla slayttakinden farklıdır. Sona bağlama adımı altyazıda net değildir (00:56:48). `gcc -Wall` ile uyarısız derlendi. İki kişi eklenip yazdırıldı, üçüncüsü bulunan düğümün arkasına eklendi, sonra bir kişi silindi; çıktılar doğruydu, AddressSanitizer bellek hatası bildirmedi. Program çıkmadan listeyi serbest bırakan döngü ayrıca eklendi.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    char name[20];
    char surname[20];
    int id;
    float salary;
} employee;

typedef struct listElement {
    employee data;
    struct listElement *next;
} listElement;

listElement *head = NULL;   /* global: hoca tavsiye etmiyor */

void printList(void) {
    listElement *p;
    int j;
    if (head == NULL) {
        printf("Liste bos\n");
        return;
    }
    for (p = head, j = 0; p != NULL; p = p->next, j++)
        printf("%d. kisi: %d %s %s %.2f\n", j + 1, p->data.id,
               p->data.name, p->data.surname, p->data.salary);
}

listElement *createListElement(void) {
    int i;
    listElement *e = (listElement *) malloc(sizeof(listElement));
    if (e == NULL) {
        printf("Bellek hatasi\n");
        exit(1);
    }
    printf("Ad soyad kimlik maas: ");
    scanf("%19s %19s", e->data.name, e->data.surname);
    scanf("%d", &i);
    e->data.id = i;
    scanf("%f", &e->data.salary);
    e->next = NULL;
    return e;
}
```

```c
void addListElement(void) {
    listElement *p;
    listElement *e = createListElement();
    if (head == NULL) {
        head = e;
        return;
    }
    p = head;
    while (p->next != NULL)
        p = p->next;
    p->next = e;
}

listElement *findElement(const char *name) {
    listElement *p;
    for (p = head; p != NULL; p = p->next)
        if (strcmp(p->data.name, name) == 0)
            return p;
    return NULL;
}

void deleteElement(listElement *gunner) {
    listElement *p;
    if (gunner == head) {
        head = gunner->next;
        free(gunner);
        return;
    }
    for (p = head; p != NULL && p->next != gunner; p = p->next)
        ;
    if (p == NULL) {
        printf("Eleman bulunamadi\n");
        return;
    }
    p->next = p->next->next;
    free(gunner);
}

void insertAfter(listElement *p, listElement *q) {
    if (p == NULL || q == NULL || p == q || q->next == p)
        return;
    p->next = q->next;
    q->next = p;
}

int main(void) {
    char ad[20];
    listElement *q, *yeni;
    addListElement();
    addListElement();
    printList();
    yeni = createListElement();
    printf("Hangi addan sonra eklensin? ");
    scanf("%19s", ad);
    q = findElement(ad);
    if (q != NULL)
        insertAfter(yeni, q);
    else
        free(yeni);
```

```
    printList();
    printf("Silinecek ad: ");
    scanf("%19s", ad);
    q = findElement(ad);
    if (q != NULL)
        deleteElement(q);
    printList();
    while (head != NULL)
        deleteElement(head);
    return 0;
}
```

11. Yapı Hizalaması (Structure Alignment)

Hoca yapı üyelerinin bellekte yan yana durmayabileceğini anlattı. Önce bir örnek verdi: `char` 1 bayt, `short` 2 bayt, yine `char` 1 bayt. İlk yapı anlatılırken bunun 4 bayt olduğu düşünülmüştü. Ama `sizeof` daha fazla çıkabilir. Çünkü derleyici yeri tahsis ederken üyeleri belli bir düzene oturtur, aradaki boşluklar başka bir yapı tarafından kullanılamaz. Hoca bunu "sanki 8 baytlık yer verilmiş gibi, halbuki 4 bayt veriliyor" diye anlattı. Buna yapı hizalaması denir.

Hoca iki durumu çizerek karşılaştırdı. Hizalama kısıtı olmayan durumda `char`, `short`, `char` art arda dizilir. Hizalama olan durumda `char`'ın arkasında bir baytlık boşluk oluşur ve bu boşluk yapının altı bayt tutmasına yol açar. Bu kısıtları açıp kapatan derleyici seçenekleri vardır, ama her işletim sisteminde ya da her derleyicide işe yaramaz. Bu, programcının genel olarak uğraşacağı bir ayar değildir.

Hocanın önerisi üyeleri planlı sıralamaktır. Örneğin bir `int` (4 bayt), iki `char` (toplam 6 bayt) ve 10 elemanlı bir `int` dizisi (40 bayt) olan yapıda sıralamayı değiştirmek bazen bellek açısından fayda sağlar.

> **Ders sonrası not:** Hoca "belleklerimiz 8 baytlık hücreler" dedi ve yapıyı 8'in katlarına göre düşünmeyi önerdi. Aşağıdaki denemeler sonucun 8'in katı olmak zorunda olmadığını gösterir. Kural büyük olasılıkla şudur: Her üye kendi hizalama gereksinimine uygun adreste başlar, yapının boyutu da en büyük hizalamanın katına yuvarlanır. Aşağıdaki sonuçlar gcc 15.2 ile 64 bit Linux'ta `sizeof` ile denendi:

Yapı	sizeof (bayt)
`char, short, char`	6
`char, char, short`	4
`char, short, char` (hizalama kapalı, `packed`)	4
`char, short, char, int`	12
`int, short, char, char`	8
`int, char, char, int[10]`	48
`int[10], int, char, char`	48
`char, int, char, int[10]`	52

> **Ders sonrası not:** Tablo hocanın iki sayısını da netleştirir. `char, short, char` için "altı bayt" doğrudur, "8 bayt" ise bu yapı için çıkmaz. `int`, iki `char` ve `int[10]` örneğinde 40 baytlık diziyi öne almak tek başına kazandırmaz, iki `char` yan yana olduğu sürece boyut 48'dir. Kazanç, küçük üyeleri bölen dizilimlerden kurtulunca gelir.

12. Bit Alanları (Bit Field)

Hoca bit alanlarını yapı hizalamasından daha faydalı bulduğunu söyledi. Normalde bir karaktere 1 bayt, yani 8 bit ayrılır. Cinsiyet bilgisi saklanacaksa ve değer 0 ya da 1 ise aslında 1 bit yeter. Oysa karakter olarak yazılan 0 ve 1, ekranda tek işaret gibi görünse de ASCII tablosundan dolayı 8 bit yer tutar. Yapı üyelerinde bit alanı kullanılarak bir üyenin daha az yer tutması sağlanır.

Hocanın ikinci örneği, 0 ile 10 bin arasında değer alan bir üyedir. `int` 32 bit yer tutar, oysa bu kadarı gerekmez. Bit alanı, istenen kadar yeri kullanıp kalan yeri başka üyelere bırakmayı mümkün kılar.

> **Ders sonrası not:** Hoca bu değer için "short'a sığmıyor, short'un biraz daha fazlası lazım" dedi. Python ile hesaplandı: 10000 için 14 bit yeter ve `short` en fazla 32767 tutar, yani değer `short`'a sığar. "Fazlası gerekir" sözü bu aralık için geçerli değildir. Hoca hemen ardından "3 baytlık veri olsun mesela" diye genel bir durumdan söz etti, asıl fikir budur.

12.1 Tanım ve Bellekteki Yerleşim

Bit alanı, üyenin adından sonra iki nokta üst üste ve bit sayısı yazılarak tanımlanır. `int a` normalde 32 bit iken `int a : 5` onu 5 bite indirir.

Hocanın karşılaştırması şöyleydi. `int a; int b; int c;` üç üye 4'er bayttan 12 bayttır. Yapı 1000 adresinden başlarsa `a` 1000-1004, `b` 1004-1008, `c` 1008-1012 aralığındadır (hoca adresleri onaltılık düşünmemizi de söyledi). `int a : 3; int b : 7; int c : 2;` yazılınca ise üyeler bit cinsinden peş peşe yerleşir. Toplam 12 bit tutarlar.

> **Ders sonrası not:** gcc 15.2'de bu ikinci yapının `sizeof`'u 4 çıktı, birincisinin 12.

Bir bit alanının birkaç bellek hücresine bölünmesi de istenmez. Hoca 16 bitlik bir yerleşim çizerek şunu anlattı: 10 bit istendiğinde hücrede 10 bitlik yer yoksa, alanın bir kısmı burada, bir kısmı öbür hücrede olamaz. Tamamı sonraki hücreden verilir. Bu yüzden aralarda boşluklar oluşabilir.

12.2 Kısıtlar

İki kısıt vardır:

- **Adres alınamaz:** Bellek adresleri bayt cinsindendir, bit alanları ise bit cinsindendir. `&a`, `&b`, `&c` diye bir şey alınamaz, bir işaretçiye atanamaz, `scanf`'e doğrudan verilemez. Adres gereken yerde geçici bir değişken kullanılır.
- **Dizisi tanımlanamaz:** Bir bit alanının kendisinden dizi oluşturulamaz. Bit alanı içeren bir yapının dizisi ise tanımlanabilir.

Hoca bunu örnekle gösterdi. Bir üyesi 8 bitlik bit alanı olan yapıdan 4 elemanlı dizi tanımlamak sorun değildir. Her eleman kendi içinde bu alanı tutar. Ama dizinin ilk elemanının bu alanına `scanf` ile `&` kullanarak okuma yapılamaz. Bu doğrudan derleme zamanında hatadır: "cannot take address of bit-field".

> **Ders sonrası not:** Bu hata gcc 15.2 ile denendi: `error: cannot take address of bit-field 'data'`. Üyeyi `unsigned a : 5[4];` gibi dizi yazmak da derlenmedi.

12.3 Neden unsigned?

Hoca doğum tarihi örneğini kullandı. Gün için 5 bit, `unsigned` olarak ayrılır. Gerekçe şudur: Bitlerle çalışıldığı için alan `signed` olursa en yüksek bit işaret biti gibi yorumlanabilir.

Hoca 13 sayısını yazdı. 13'ün ikilik karşılığı `1101`'dir (önce yanlış yazdı ve düzeltti). 5 bitte `01101` olarak tutulur. Bu sayı `signed` ve `unsigned` için aynıdır, zaten pozitiftir. Ama en soldaki bit 1 olsaydı sayı negatif yorumlanabilirdi. Bu yüzden `unsigned` tercih edilir.

> **Ders sonrası not:** Hocanın "en solda 1 olsaydı" durumu gcc 15.2 ile denendi (ara adım eklendi). `11101` bit dizisi 5 bitlik `unsigned` alanda 29, 5 bitlik `int` alanda -3 okunur.

12.4 Tarih Yapısı ve Örnek

Tarih için gün 5 bit, ay 4 bit, yıl 11 bit kullanılır. Böylece bir `int` kadar yer tutacak yerde üç `int`'in bilgisi tutulabilir. Aradaki boşluklar çok önemli değildir.

> **Ders sonrası not:** Python ile hesaplandı: 5 bit en fazla 31, 4 bit en fazla 15, 11 bit en fazla 2047 değerini alır, toplam 20 bittir.

Hoca çalışan örneğini bu kez isim, kimlik numarası ve doğum bilgileriyle genişletti. Yapının geri kalanı, head ve düğümler aynı kaldı. Yazdırmada değişen bir şey yoktur, alanlar doğrudan ekrana yazılır. Fark oluşturma tarafındadır: `scanf` içinde alanın adresi kullanılamaz. Onun yerine `unsigned` türünde `value` adlı geçici değişkene okuma yapılır, sonra değer gün, ay ve yıl alanlarına atanır.

Hoca burada değer kontrolünü de hatırlattı. Girilen gün 1 ile 31 arasında mı diye bakılabilir. Daha ileri gidilip her ayın gün sayısı da kontrol edilebilir. Sağlıklı olan, önce kontrolü yapmak, sonra değeri alana kaydetmektir.

> **Ders sonrası not:** Bu program kısa nottaki 5, 4, 11 bitlik yapıyla kuruldu ve altyazıdaki `value` yöntemini izler. Girdiler ve mesajlar derstekinden büyük olasılıkla farklıdır. `gcc -Wall` ile uyarısız derlendi; 13, 5, 2004 girişiyle `sizeof` 4 ve `13.5.2004`, 40 günü girilince hata mesajı verdi.

```c
#include <stdio.h>

struct tarih {
    unsigned d : 5;
    unsigned m : 4;
    unsigned y : 11;
};

int main(void) {
    struct tarih t;
    unsigned value;

    printf("Gun: ");
    scanf("%u", &value);
    if (value < 1 || value > 31) {
        printf("Gecersiz gun\n");
        return 1;
    }
    t.d = value;

    printf("Ay: ");
    scanf("%u", &value);
    if (value < 1 || value > 12) {
        printf("Gecersiz ay\n");
        return 1;
    }
    t.m = value;

    printf("Yil: ");
    scanf("%u", &value);
    if (value > 2047) {
        printf("Gecersiz yil\n");
        return 1;
    }
    t.y = value;

    printf("sizeof = %zu\n", sizeof(t));
    printf("%u.%u.%u\n", t.d, t.m, t.y);
    return 0;
}
```

13. Birlikler (Union)

Union, yapılara çok benzer, ama üyeler aynı bellek alanını paylaşır. Tanımı `struct` gibi yapılır, yalnızca anahtar sözcük `union`'dır. `int a; char b;` yazılınca iki üyeye ayrı ayrı yer ayrılmaz. En büyüğüne yetecek kadar, yani 4 bayt yer ayrılır, 5 bayt değil. Bu alan isterseniz tam sayı

için, isterseniz karakter için kullanılır, ikisi aynı anda tutulmaz. Hocanın örneği: Kişinin kimlik numarası tutulur, yabancı uyrukluysa aynı alanda pasaport numarası tutulur.

13.1 Slayttaki Örnek

Slayttaki union, içinde `c1` ve `c2` adlı iki karakter olan bir yapı, bir tam sayı `j` ve bir `float` içerir. Bu alanda ya iki karakter ya tam sayı ya da `float` tutulur, tercih programcınındır.

`'a'` ve `'b'` girilirse bu alana 97 ve 98 sayılarının bitleri, 8'er bit olarak yazılır. Aynı bitler 16 bitlik `j` olarak okunursa başka bir sayı, `float` olarak okunursa yine başka bir sayı görülür. Ardından `j = 5` atanırsa, alanın tamamına 5'in bitleri yazılır. `a` ve `b`'nin kodları tamamen silinir (override edilir), artık onlara dair bir şey görülmez.

> **Ders sonrası not:** Hocanın "başka bir sayı" dediği değer python ile hesaplandı ve gcc ile (2 baytlık `short j` kullanılarak, x86-64) denendi (ara adım eklendi): $97 + 98 \cdot 256 = 25185$. Sonuç bayt sırasına bağlıdır.

13.2 Derste Yazılan Örnek

Hoca `sizeof` sorusuyla başladı. `int`, `char` ve `double` üyeli bir union için cevap ne olur? 13 mü, 16 mı? Çıkan cevap 8'dir, çünkü union en büyük üyesi kadar yer tutar. Üyeler `float` gibi 4 baytlık olacak şekilde değiştirilince `sizeof` 4 oldu.

> **Ders sonrası not:** Bu iki değer gcc 15.2 ile de denendi: `int, char, double` için 8, `int, char, float` için 4.

Sonra union türünden `n` tanımlandı ve `n.b = 'A';` atandı. Burada `malloc` gerekmez, her şey doğrudan kullanılabilir.

Hoca üç sorunun cevabını aradı:

- **Neden nokta?** Çünkü `n` bir işaretçi değildir, yapı gibi doğrudan bir değişkendir.
- **Neden 'A' tırnakla?** İlk yazımda atama hatalıydı ve `A` beklendiği gibi görünmedi. `'A'` yazılarak düzeltildi.
- **`node *n; n->b = ...` yazılsaydı olur muydu?** Hayır.

Son soru önemlidir. İşaretçinin görevi bir yeri göstermektir. Burada `n` henüz hiçbir yeri göstermez. Önce `malloc` ile ya da bir değişken tanımlayarak yer açılmalıdır:

```
node l;
node *n;
n = &l;
n->b = 'A';
```

Hocanın benzetmesi şuydu: "Bankada altın hesabın var, altın alıyorsun ama öyle bir altın yok." Önce fiziki olarak oluşturulacak, sonra işaretçiyle gösterilecektir. Hoca bu adımın öğrenciler tarafından sık atlandığını da söyledi.

Çıktı `4 A 65` biçiminde oldu: `sizeof` 4, karakter olarak okunan `b` `A`, sayı olarak okunan değer 65. Sonra `c` alanı da yazdırıldı, `float` olarak `0.000000` göründü. Hoca bunun sebebini bitlerin çoğunun sıfır olmasıyla açıkladı.

Sonra sınıfa şu soruyu sordu: Tam sayı alanına hangi değeri yazarsam karakter alanından küçük `a` okurum? Bir cevap 65 oldu, ama o büyük `A`'dır. Doğrusu 97'dir. `n.a = 97;` yazıldıktan sonra aynı kod çalıştırılınca `b` küçük `a` olarak görüldü. Hoca bunu şöyle anlattı: 32 bitlik alanın tamamı 97'ye göre yeniden yazılır, `b` okunurken alanın ilk kısmı alınır ve 97, küçük `a` olarak basılır.

> **Ders sonrası not:** `n.b = 'A'` yalnızca bir baytı değiştirir. Diğer üç bayt ilk değer verilmemiş yerel değişkenden kalan değerdir. Derste 65 görülmesi bu baytların sıfır olmasına bağlıdır, garanti değildir. Hoca da "bellek hücresinin sıfırlarla dolması lazım" dedi. Aşağıdaki kodda bu yüzden önce `n.a = 0;` yazıldı (ara adım eklendi).

> **Ders sonrası not:** `float` alanı gerçekte sıfır değildir. Python ile hesaplandı: 65'in bitleri `float` olarak yaklaşık 9.1×10^{-44}, 97'nin bitleri yaklaşık 1.36×10^{-43} eder. `%f` bunları `0.000000` diye yuvarlar, `%g` gösterir.

Ek bilgi (derste anlatılmadı): Hocanın “ilk kısım” dediği bayt, büyük olasılıkla küçük uçlu (little-endian) işlemcilerde sayının en düşük anlamlı baytıdır. x86-64 üzerinde denendi: `b`, tam sayının en düşük 8 bitini okudu. Büyük uçlu bir makinede aynı kod büyük olasılıkla farklı sonuç verir. Bu yüzden bir üyeye yazıp başka üyeden okumanın anlamı büyük olasılıkla uygulamaya bağlıdır.

Ders sonrası not: Program derste yazılanın kısa nottaki hâlinden kuruldu. `gcc -Wall` ile uyarısız derlendi, çıktı `4 A 65 9.10844e-44` ve `a 97 1.35926e-43` oldu.

```c
#include <stdio.h>

typedef union {
    int a;
    char b;
    float c;
} node;

int main(void) {
    node n;

    n.a = 0;
    n.b = 'A';
    printf("%zu %c %d %g\n", sizeof(n), n.b, n.a, n.c);

    n.a = 97;
    printf("%c %d %g\n", n.b, n.a, n.c);
    return 0;
}
```

13.3 Soru: Union Üzerinden Sayı ve Karakter

Soru: Kullanıcıdan 10000 gibi bir sayı alınsa, union içindeki karakter alanları ile bu sayının basamakları okunabilir mi? **Cevap:** Hayır. Yapıdaki iki karakterin ayrıldığı örnekte 4 baytlık alanın iki baytı iki karakter içindir. 10000 bir karakter olamaz, basamaklarını tutmak için ancak 5 karakterlik bir dizi gerekir. Dahası, bu sayının ikilik karşılığı ayrı bir şeydir. 32 bitin başı sıfırlarla dolar, son bitleri ise sayının değerine göre 0 veya 1’dir. Bu bitlerin bir kısmı karakter alanından okunabilir, ama bunlar 10000 yazısındaki `1` ve `0` karakterleri değildir. Basamaklar isteniyorsa ASCII tablosundan çevrilmesi gerekir. String olarak işlemek daha kolaydır.

Ders sonrası not: Hoca bu cevapta 10000 için “15 bit lazım” dedi. Python ile hesaplandı: 10000’in ikilik karşılığı `10011100010000`’dır, 14 bit gerekir.

13.4 TC Vatandaşı Örneği

Slayttaki son örnek, aynı çalışan listesine vatandaşlık bilgisini ekler. Kişi Türkiye Cumhuriyeti vatandaşıysa doğum şehri, değilse uyruk tutulur. Bu iki bilgi ayrı ayrı 20’şer karakter olarak tutulursa 40 bayt gider. İkisi `union` altına alınırsa 20 bayt yeter. Kişinin vatandaş olup olmadığını gösteren 1 bitlik bir bayrak alanı tutulur ve program buna göre seçim yapar: Bayrak 1 ise doğum şehri alanına, değilse uyruk alanına yazar.

Ders sonrası not: Hoca bu örnekteki tarih alanlarından “16-18 bitlik”, sonra “11, 16, 18 bitlik” yer diye söz etti. Python ile hesaplandı: 5, 4 ve 11 bitin toplamı 20 bittir.

Hoca iki alanın da aynı türde ve aynı boyutta olmasının bu örneği çok anlamlı kılmadığını kabul etti. Asıl uygun kullanım örneği, pasaport numarası ile TC kimlik numarası gibi seçimlerdir.

Soru: Bit alanlarında da fazladan yer tutulmuyor mu? **Cevap:** O ayrı bir mesele, bit alanlarında bu açıdan bir değişiklik yok, o alan zaten yer tutacak. Burada union’dan söz ediyoruz. Bir kişi aynı anda iki durumda da olamayacağı için, yapıyla yazılınca 20 baytlık alan gereksiz yere ayrılır, union bunun önüne geçer.

Hoca aranın ardından, bit alanı ve union kullanımında fazla zorluk olmadığını, belli kuralları bilmenin yeterli olduğunu ve sonraki konularda başka örnekler yapılacağını söyledi.

14. Özyinelemeye Giriş: Faktöriyel

Hoca özyinelemeyi "farklı bir konu ama çok çok lazım olan bir konu" diye tanıttı. Başlangıç noktası, $n!$ hesaplayan bir fonksiyondu. Önce döngüyle yazıldı:

```
int f(int n) {
    int r = 1, i;
    if (n < 0)
        return -1;
    if (n == 0 || n == 1)
        return 1;
    for (i = 1; i <= n; i++)
        r = r * i;
    return r;
}
```

Negatif sayının faktöriyeli olmadığı için `-1` döndürülür. Faktöriyelin mantığı kolay olduğu için bu kod sorunlu değildir. Ama hoca, n. Fibonacci sayısını bulan bir kod istense işin karışacağını söyledi. Burada matematik ve programlama devreye girer. Bazı problemleri doğrudan döngü yazmadan hesaplayabilir miyiz diye düşünülmüştür. Buna özyineleme denir.

Matematikten şu bilinir: $n! = n \cdot (n-1)!$. $(n-1)!$ yerine de $(n-1) \cdot (n-2)!$ yazılabilir. Açılım şöyle sürer:

$$n! = n \cdot (n-1) \cdot (n-2) \cdots 1!$$

Son terim $1!$ ya da $0!$'dir ve ikisinin de 1 olduğu bilinir. Yani sonucu bilinen bir temel çözüm vardır. Diğer çözümler bundan üretilir. Bu temel çözüme temel durum (base case) denir. Kod şöyle olur:

```
int f2(int n) {
    if (n == 0 || n == 1)
        return 1;
    return n * f2(n - 1);
}
```

`f2`, kendi içinden kendi adıyla tekrar çağrılmıştır. Özyineleme (recursion), kendini tekrar ettirmek demektir. Temel durum tanımlanır, kendini tekrar edecek çözüm de fonksiyon tekrar çağrılarak yazılır.

14.1 Çağrıların İzlenmesi

Hoca `f2(3)` çağrısını bellekte izledi (önce 5 dedi, sonra kısa olsun diye 3 aldı).

1. Program açılır, stack bölgesine `main` yerleşir. `main`, `f2(3)`'ü çağırır.
2. `n` 0 ya da 1 değildir. `3 * f2(2)` hesaplanacaktır. `f2(3)`, `f2(2)`'nin sonucunu bekler, `main` de `f2(3)`'ün sonucunu bekler.
3. `f2(2)`: `n` 0 ya da 1 değildir. `2 * f2(1)` hesaplanacaktır, `f2(1)` beklenir.
4. `f2(1)`: `n` 1'dir, temel duruma girilir, 1 döndürülür.
5. `f2(2)`: $2 \cdot 1 = 2$ döndürür. Hoca burada sıraya dikkat çekti.
6. `f2(3)`: $3 \cdot 2 = 6$ döndürür ("2 çarpı 3 değil, 3 çarpı 2").
7. `main` 6 değerini alır ve ekrana basar.

Hocanın benzetmesi matematikteki parantezlerdir. Üstü köşeli, sonra yuvarlak parantez açıldığında, en son açılan ilk kapanır, ilk açılan en son kapanır. Burada da en son çağrılan `f2(1)` ilk tamamlanır, onun sayesinde `f2(2)`, onun sayesinde `f2(3)` tamamlanır, sonra `main`'e dönülür.

Hoca sonuçta "sonsuza gitmeme" meselesine de değindi. Bir fonksiyon başka bir fonksiyonu çağırabildiğine göre kendini de çağırabilir. Ama çağrı sürekli devam ederse sonsuza giden bir durum olur. Sonsuzluğu durduran ve döngüden çıkaran şey baştaki temel durumdur.

14.2 İki Çözümün Karşılaştırılması

Çalışma zamanı açısından bu örnekte çok bir fark yoktur, ikisinde de yaklaşık n çarpma yapılır. Kodun kısalığı ve temizliği açısından özyinelemeli çözüm daha iyidir. Bellek açısından ise daha kötüdür. Döngülü sürüm çağrıldığında stack'te `n`, `i` ve `r` için yer açılır, hesap biter, geri dönülür. Özyinelemeli sürüm ise stack bölgesini iç içe çağrılarla gereksiz yere doldurur.

Hoca ayrıca iki çözümün yönünü karşılaştırdı. Döngüde ileri doğru gidilir: $1 \cdot 1$, sonra $2 \cdot 1$, $3 \cdot 2$, $4 \cdot 6$ ve böyle sürer. Sonuç her adımda `i` ile çarpılır. Özyinelemede geriye doğru toparlanır: Önce en içteki 1 elde edilir, sonra onunla 2, sonra onunla 3 çarpılır.

15. Doğrudan ve Dolaylı Özyineleme

Fonksiyonun kendi içinden kendini çağırmasına doğrudan özyineleme (direct recursion) denir. Dolaylı özyineleme (indirect recursion) ise bir zincirle olur: `x` fonksiyonu `y`'yi, `y` fonksiyonu `z`'yi, `z` fonksiyonu tekrar `x`'i çağırır. `x`'ten `y`'ye, `y`'den `z`'ye, `z`'den tekrar `x`'e gelinmiş olur.

16. Stack Overflow Nasıl Oluşur?

Hoca, işin arkasındaki donanımsal mekanizmayı iyi bilmek gerektiğini söyledi. Çünkü hepimizin bildiği bir site adı buradan türemiştir: stack overflow.

Örnek olarak `int a` ve `int b` alan, içinde `int i` bulunan, `i` döndüren ve bir yerde kendini tekrar çağıran bir `f` fonksiyonu aldı. `main`'in 10. satırında `f(2, 3)` çağrılsın.

`f` tamamlandığında programın 11. satırdan devam etmesi gerekir. Bu yüzden 10. satırda kalındığı bilgisi stack bölgesinde tutulur. `f`'e gönderilen argümanlar da alt programda kullanılacağı için orada tutulur. `f`, içinde `a`'yı bir azaltıp `b`'yi bir artırarak kendini çağırırsa `f(1, 4)`, oradan `f(0, 5)` çağrılır. Temel durum yazılmadığı için bu böyle sürer ve stack sürekli büyür.

Hoca belleği yeniden çizdi: Bir tarafta stack, bir tarafta heap. Stack büyüdükçe heap bölgesine baskı uygular, yani taşmaya başlar. Hoca bunu, prosesin işletim sistemine "heap bölgeme müdahale ediliyor" diye sinyal göndermesi ve işletim sisteminin devreye girip programı kapatması olarak anlattı. Buna stack overflow denir.

> **Ders sonrası not:** Sinyalin yönü tersidir: İşletim sistemi prosese sinyal gönderir. Bu derste yazılan türden sonsuz çağrılı program gcc 15.2 ile Linux'ta denendi: `Segmentation fault` ile kapandı, çıkış kodu 139 oldu. Bu kod, işletim sisteminin gönderdiği 11 numaralı sinyalle kapanma demektir ($128 + 11$). Stack'in heap'e fiilen değmesi büyük olasılıkla şart değildir, sınır büyük olasılıkla işletim sisteminin prosese verdiği stack boyutuyla belirlenir.

Bu bir dezavantajdır, ama avantajları da çoktur ve uygulamalarda görülecektir. Sonsuza gitmemek için yapılacak şey bir yerde durmaktır. $n!$ hesaplanıyorsa $1!$ ya da $0!$ bilinmelidir. n'den $n-1$ çağrılır, ondan $n-2$ çağrılır. Negatife gitmemesi için sıfırda durmak gerekir. Temel durum budur.

Hoca bu sırada, biraz önce döngüyle listenin son düğümüne gidilen işin özyinelemeyle de yapılabileceğini söyledi. Bakılan düğüm `NULL` ise dönülür, değilse fonksiyon `head->next` ile bir daha çağrılır. Bu kodun sonraki derste yazılacağını belirtti.

17. Stack'i Deneyle Görmek

Hoca temel durumu olmayan basit bir `f` yazdı ve "Bu fonksiyon kaç defa çağrılır?" diye sordu. Sonsuz değil, stack dolana kadar çağrılır. Bu da belleğe, işletim sistemine ve parametrelere göre değişir. Sonra ikinci soruyu sordu: Fonksiyona durup dururken 10 elemanlı bir `int` dizisi eklense hangisi daha çok çağrılır? Dizisiz olan, çünkü her çağrıda daha az yer harcar.

Yerel değişken olmasa bile stack'e bir şeyler yazılır: kalınan satırı gösteren program sayacı (program counter), işlemcinin bayrakları ve durum yazmaçları, o ana kadarki hesaplarla ilgili bilgiler. Dizi eklenince ayrıca 40 bayt tutulur, yani bellek "kırkar kırkar" dolar. Stack'te yalnızca yerel değişken bulunduğu düşünülmemelidir.

Çağrı sayısını görmek için hoca bir sayaç ekledi. Yerel `i = 1` yazıp `i`'yi artırmak işe yaramaz, çünkü her çağrının kendi `i`'si vardır. Gerçekten de program çalıştırılınca ekranda sürekli "1. f fonksiyonu" yazdı. Çözümün iki yolu vardır:

- **`static` değişken:** Heap ya da stack'te saklanmaz, ayrı bir yerde tutulur. İlk değer bir kez verilir, sonraki çağrılar kaldığı değerden devam eder.
- **Global değişken:** O da olur, ama "global değişkeni pek sevmiyoruz".

```c
#include <stdio.h>

void f(void) {
    static int i = 1;
    printf("%d. f fonksiyonu\n", i);
    i++;
    f();
}

int main(void) {
    f();
    return 0;
}
```

Hocanın makinesinde program yaklaşık 43100 çağrıda durdu. Hoca bunu yorumlarken çağrı başına yaklaşık 10 bayt varsaydı ve 43 bin çarpı 10 baytlık bir alan olduğunu söyledi. Yalnızca çalışan bir proses için bu kadar yer verildiğini, makinede başka prosesler de olduğu için daha fazlasına izin verilmediğini belirtti. Sonra 10 elemanlı diziyi ekledi: Çağrı sayısı neredeyse yarıya, 21 bine düştü.

> **Ders sonrası not:** Program `gcc -Wall` ile derlendi. gcc `-Winfinite-recursion` uyarısı verdi, bu örnekte beklenen bir uyarıdır. Çalışınca çıkış kodu 139 ile kapandı. Hoca çağrı başına 5-10 bayt dedi, ama bu sayılar deneyle uyuşmaz. Çağrı başına s bayt harcandığını ve dizinin 40 bayt eklediğini kabul edersek $43000 \cdot s = 21000 \cdot (s + 40)$ olur. Python ile hesaplandı: $s \approx 38$ bayt. 10 bayt doğru olsaydı dizi eklenince çağrı sayısı yarıya değil 8600'e düşerdi. Ayrıca sayaç `static` olduğu için, hocanın da söylediği gibi stack'te tutulmaz, oysa hoca hesapta onu da saydı.
>
> **Ek bilgi (derste anlatılmadı):** Çağrı sayıları büyük olasılıkla Windows'ta MinGW derleyicisinin varsayılan 2 MiB'lik stack sınırıyla açıklanır (bu sınır derste geçmedi, makinede doğrulanmadı). Python ile hesaplandı: $2097152/43100 \approx 48.7$ bayt ve $2097152/21000 \approx 99.9$ bayt. Dönüş adresi 64 bit sistemde büyük olasılıkla 8 bayttır, derleyici büyük olasılıkla kayıtlı yazmaçlar ve hizalama için de yer ayırır. Hocanın dediği gibi sınır işletim sistemine göre değişir, bu yüzden aynı deney başka makinede farklı sayı verir.
>
> **Hocanın vurgusu:** Sürekli çağrılan bir fonksiyon sonsuza kadar çalışacak şekilde bırakılmaz. Bir temel durumu olmalıdır.

18. Fibonacci: Döngüyle ve Özyinelemeyle

Fibonacci sayıları 1, 1, 2, 3, 5, 8 diye gider, her terim kendinden önceki iki terimin toplamıdır. Hoca bu örneği "çok ibretlik" diye nitelendirdi. Bilgisayar biliminde beş altı farklı Fibonacci çözümü vardır, bazısı iyi, bazısı çok kötüdür. Burada anlatılan özyinelemeli çözüm neredeyse en kötülerindendir. Ama konuyu öğretmek için güzeldir, kod verimliliği açısından ise çok kötüdür.

18.1 Döngüyle Yazım

İlk iki terim bilinir: `s1 = s2 = 1`. Kalan terimler için döngü döner. Hoca döngüyü, üç değişkenin kendi içinde kaydırıldığı bir dairesel takas gibi anlattı. İlk yazımda sonuç baştan `r = s1 + s2` ile hesaplanıp döngüde `s1 = s2; s2 = r; r = s1 + s2;` yazıldı. Beşinci terim istenince 13 çıktı. Hoca önce indisleri sayıp bunu doğru buldu.

Özyinelemeli sürümle sonuçlar uyuşmayınca kod gözden geçirildi. Önce `s1` ile `s2` toplanıp sonra kaydırma yapılmasına karar verildi.

> **Ders sonrası not:** İlk çıktı 13'tür. Python ile izlenince $i = 2, 3, 4, 5$ için `r` sırasıyla 3, 5, 8, 13 olur. İndis sıfırdan başlayınca dizi $F_0 = 1, F_1 = 1, F_2 = 2, F_3 = 3, F_4 = 5, F_5 = 8, F_6 = 13$ olduğu için 13, beşinci değil altıncı indistir. Yani hocanın "5. ise 13, doğru" sayımı bir fazladır. Düzeltilmiş sürüm $F_5 = 8$ verir.

18.2 Özyinelemeyle Yazım

Özyinelemeli sürümde tek bir temel durum yetmez, iki terim gerekir. Çünkü ilk iki terim bilinip üçüncüye öyle gidilir. Hoca önce `n == 0`, `n == 1` ve `n == 2` için 1 döndürdü. Sınıfta sonucun uyuşmadığı konuşuldu ve temel durumlar `n == 0` ve `n == 1` olarak bırakıldı. Gövde şudur: Bir önceki ve iki önceki terimi ayrı ayrı hesapla, topla, döndür. Faktöriyelden farklı olarak fonksiyon burada iki kez çağrılır, ikili bir ağaç gibi dallanır. Hoca kodu çok kısa sürede yazdığına, ayrı bir mantıkla uğraşmadığına dikkat çekti.

> **Ders sonrası not:** Aşağıdaki program, altyazıda sözlü anlatılan iki sürümün son hâlinden kuruldu. `fib` içinde `r`'nin ilk değeri ($n < 2$ için de doğru olması için) ayrıca verildi. `gcc -Wall` ile uyarısız derlendi. `fib(5)` ve `fib2(5)` 8, `fib2(4)` 5 verdi.

```c
#include <stdio.h>

int fib(int n) {
    int s1 = 1, s2 = 1, r = 1, i;
    for (i = 2; i <= n; i++) {
        r = s1 + s2;
        s1 = s2;
        s2 = r;
    }
    return r;
}

int fib2(int n) {
    if (n == 0)
        return 1;
    if (n == 1)
        return 1;
    return fib2(n - 1) + fib2(n - 2);
}

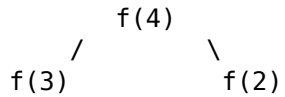
int main(void) {
    printf("%d %d %d\n", fib(5), fib2(5), fib2(4));
    return 0;
}
```

18.3 F(4) Çağrı Ağacı

Hoca `f(4)`'ü adım adım izledi. Fonksiyon çağrısı toplamadan önce tamamlanmalıdır. Bu yüzden `f(3)` bitmeden sağdaki `f(2)`'ye geçilmez.

1. `f(4)`, `f(3)`'ü çağırır. `f(3)`, `f(2)`'yi çağırır. `f(2)`, `f(1)`'i çağırır.
2. `f(1)` 1 döndürür. Sonra `f(2)`, `f(0)`'ı çağırır. O da 1 döndürür. `f(2)` $1 + 1 = 2$ döndürür.
3. `f(3)`, kendi `f(1)`'ini çağırır, 1 gelir. `f(3)` $2 + 1 = 3$ döndürür.
4. `f(4)`, sağdaki `f(2)`'yi çağırır. O da `f(1)` ve `f(0)`'ı çağırır, $1 + 1 = 2$ döndürür.
5. `f(4)` $3 + 2 = 5$ döndürür.

Ağaç şöyledir:



```
          /    \          /    \
      f(2)     f(1)  f(1)    f(0)
     /    \
 f(1)     f(0)
```

Dizi 1, 1, 2, 3, 5 olduğu için, sıfırdan sayılınca dördüncü değer 5'tir.

18.4 Maliyet ve Dinamik Programlama

Hoca iki kötülük saydı. Birincisi, n büyüdükçe ağaç hızla büyür. $n = 4$ için çağrıları saydı: 9 tane. Yaklaşık olarak 2^{n-1} kadar işlem olduğunu söyledi. $n = 4$ için $2^3 = 8$ eder. n bir artınca $2^4 = 16$ olur, yani iş 2 katına çıkar. $n = 40$ gibi bir değerde ağaç devasa olur.

Ağacın derinliğine de baktı: En sol dal `f(n)`, `f(n-1)`, `f(n-2)` diye yaklaşık n adım iner. Her alt dal da kendi içinde derinleşir. Yani n'den fazla dal vardır. 2^{n-1} karmaşıklığının bir sebebi budur. Bunun ispatının ayrık matematik dersinde, kodlamalarının algoritma analizi dersinde ele alınacağını söyledi.

> **Ders sonrası not:** Sayım python ile yapıldı. Çağrı sayıları $n = 0, 1, \ldots, 8$ için 1, 1, 3, 5, 9, 15, 25, 41, 67'dir. Hocanın 9 sayısı doğrudur. Ama n bir artınca çağrı sayısı tam 2 katına çıkmaz: $15/9 \approx 1.67$ ve büyük n'de oran yaklaşık 1.618'e iner. $n = 40$ için çağrı sayısı 331160281'dir, $2^{39} = 549755813888$ ise bunun yaklaşık 1660 katıdır. Yani 2^{n-1} gerçek değeri değil, yukarıdan bir sınırı verir. Üstel büyüme fikri doğrudur.

İkinci kötülük tekrardır. Aynı ağaçta `f(0)` ve `f(1)` birkaç defa hesaplanır, bunlar gereksizdir. Bunun çözümü dinamik programlamadır (hocanın andığı adıyla memoization). Bu, veri yapılarında gösterilecek bir algoritma geliştirme tekniğidir. Bir değerin sonucu bir kez hesaplanınca diziye yazılır. Tekrar gerektiğinde hesaplamadan diziden alınır. Hesap maliyetinden kurtulunur, ama bunun için bellek harcanır. Hoca döngüyü diziyle yazmanın da dinamik programlamaya girdiğini söyledi.

Hoca bu örneklerin amacını şöyle özetledi: Şu an yalnızca özyinelemeli fonksiyonun C'de nasıl yazılacağına ve kurallarına bakılıyor.

Slayttaki çağrı çizimlerinin de aynı mantığı gösterdiğini söyleyip parametrelerin, işlemci durumunun, carry ve zero gibi bayrakların stack'te saklandığını tekrar vurguladı.

19. Üs Alma: Özyinelemenin İşe Yaradığı Bir Örnek

Faktöriyelde kod kısaldı, Fibonacci'de zaman kaybı oluştu. Peki özyinelemenin gerçekten işe yaradığı bir yer var mı? Hoca x^n hesabını örnek verdi. Bunun için n defa döngü çevirmeye gerek yoktur. $x^{n/2}$ bulunup kendisiyle çarpılırsa x^n elde edilir. $x^{n/2}$ için de $x^{n/4}$ bulunur. Özyinelemeli çağrıda üs yalnızca birer birer azalmak zorunda değildir, $n/2$ de yazılabilir. Böylece n, $n/2$, $n/4$, $n/8$ diye x^1'e doğru gidilir.

Hoca, özyinelemeli bir fonksiyonda en önemli şeyin temel durumu doğru tespit etmek olduğunu söyledi.

> **Hocanın vurgusu:** Temel durum doğru tespit edilirse aşağıdaki dallanmalar zaten kendi içinden onu çağırır.

Temel durumlar şunlardır: `y == 0` ise 1 döndür, `y == 1` ise `x` döndür. Sonra `mypower(x, y/2)` çağrısı kendisiyle çarpılır. Örnek olarak önce 3 üzeri 45 istendi, sonra daha küçük bir üsle, 3 üzeri 4 ile denendi.

Hoca, klasik yöntemle n çarpma yapılırken bu yolla $\log_2 n$ çarpmanın yeteceğini söyledi. 2^{10} için 10 işlem yerine 3 işlem, 2^{1024} için 1024 işlem yerine 10 işlem. Ama kodda bir eksik olduğunu ve sınıfın bunu düşünmesini istedi. İki çağrı aynı şeyi hesaplar. Fibonacci'de iki çağrı farklıydı ($n - 1$ ve $n - 2$), yapacak bir şey yoktu. Burada ise biri biliniyorsa sonuç onunla çarpılır, tekrar fonksiyon çağrısı yapılmaz.

Soru: Üs ikinin kuvveti olmazsa çalışmıyor. 3 üzeri 4 tamam, 3 üzeri 5'te de 81 çıkıyor, neden?
Cevap: Bu, tam sayı bölmesiyle ilgilidir. Tek üslerde ayrı bir şey yapılacak. Şu an sonucun doğruluğuyla ilgilenmeyin, devamı sonraki derste.

Ders sonrası not: Sayılar python ile hesaplandı. Derste yazılan iki çağrılı sürümde `mypower(3, 5)` $= 81$ olur (doğrusu 243). Çağrı sayısı da düşmez: $y = 1024$ için 2047 çağrı ve 1023 çarpma yapılır. Tek çağrılı sürümde aynı üs için 10 çarpma yeter. 2^{10} için hocanın 3 sayısı tam değildir, $10 \to 5 \to 2 \to 1$ yolunda tek üs ek bir çarpma gerektirdiği için 4 çarpma yapılır. Ayrıca $3^{45} = 29543127065508336986 43 \approx 2.95 \times 10^{21}$ ne `int` (en fazla 2147483647) ne de `long long` (en fazla 9223372036854775807) içine sığar, sonuç taşar.

Ders sonrası not: Aşağıda hem derste yazılan sürüm hem de düzeltilmiş sürüm (`fastpow`, derste yazılmadı, kısa nottan uyarlandı) vardır. `gcc -Wall` ile uyarısız derlendi. Çıktı `81 81 243 81 1024` oldu.

```c
#include <stdio.h>

int mypower(int x, int y) {
    if (y == 0)
        return 1;
    if (y == 1)
        return x;
    return mypower(x, y / 2) * mypower(x, y / 2);
}

long long fastpow(long long x, int y) {
    long long h;
    if (y == 0)
        return 1;
    if (y == 1)
        return x;
    h = fastpow(x, y / 2);
    return (y % 2) ? h * h * x : h * h;
}

int main(void) {
    printf("%d %d ", mypower(3, 4), mypower(3, 5));
    printf("%lld %lld %lld\n", fastpow(3, 5), fastpow(3, 4), fastpow(2, 10));
    return 0;
}
```

Dersin Toparlaması

Bu derste bağlı liste kodla kuruldu. Düğümlerin neden `malloc` ile heap'te oluşması gerektiği, `next`'in neden `NULL` yapılması gerektiği, sona eklemenin neden son düğümde durması gerektiği görüldü. `deleteList` örneği, parametre aktarımında fonksiyon içindeki `head` değişikliğinin `main`'e yansımadığını çökmeyle gösterdi. Çözüm olarak yeni değeri döndürmek ve ikinci dereceden işaretçi anıldı. Slayttaki çalışan örneği ile silme, arama ve araya ekleme işlemleri okundu, global `head`'in neden tavsiye edilmediği konuşuldu. Ardından yapı hizalaması, bit alanları ve union ile bellek kullanımını kontrol etmenin yolları görüldü.

İkinci yarıda özyineleme anlatıldı. Temel durum ve kendini çağıran adım, faktöriyel, çağrıların stack'teki izlenmesi, stack overflow ve Fibonacci'nin maliyeti ele alındı. Üs alma örneği bir sonraki derse köprü oldu: `mypower` kodundaki tek üs ve tekrar eden çağrı sorunu orada tamamlanacak. Listenin özyinelemeyle gezilmesi ile `void` bir silme fonksiyonunun ikinci dereceden işaretçiyle yazılması da sonraki derse bırakıldı.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.